

Avance del Proyecto Final — Pregunta 6

Manejo del Repositorio en GitHub y Estructura de Ramas

Datos del Estudiante

Estudiante: Brando Enrique Chávez Vergara (brandon.chavez@udea.edu.co)
Proyecto: Nodo de Edge Computing y Gateway IoT para Monitorización Industrial
Docente: Ricardo Andrés Velásquez Vélez

1. Enlace al Repositorio

Repositorio GitHub Oficial

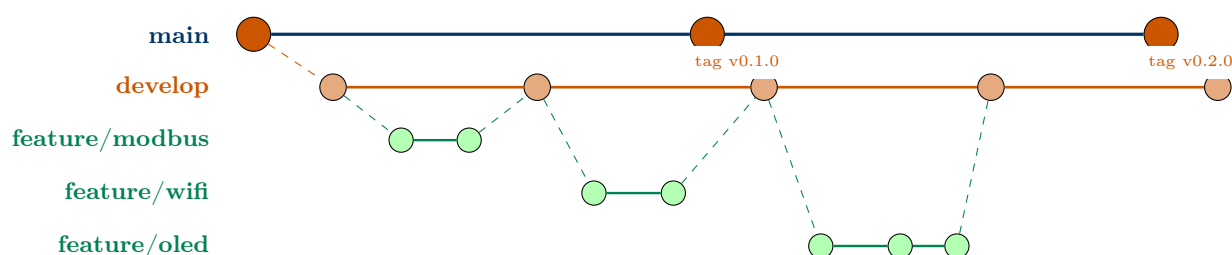
https://github.com/kiikechavez/IoT_Edge_Gateway

2. Estrategia de Manejo del Repositorio

Se adopta el modelo **Git-Flow simplificado**, recomendado para proyectos embebidos individuales o de equipos pequeños. Los principios:

- La rama **main** **siempre** contiene código compilable y verificable.
- Todo desarrollo se realiza en ramas **feature/*** aisladas.
- La rama **develop** integra cambios antes de pasar a **main**.
- Cada **feature/*** se mergea vía *Pull Request* con revisión (auto-revisión en este caso, al ser proyecto individual).
- Versionado semántico mediante tags (**v0.1.0**, **v0.2.0**, ...).

3. Estructura de Ramas (Branching Model)



3.1. Listado de Ramas

Rama	Tipo	Propósito
main	permanente	Código estable y liberable. Solo recibe merges desde develop vía PR.
develop	permanente	Rama de integración continua. Aquí convergen los feature/* .

Rama	Tipo	Propósito
feature/modbus-driver	temporal	Driver Modbus RTU (UART0 + DMA + timer 3.5 char).
feature/wifi-http-client	temporal	Wi-Fi CYW43439 + cliente HTTP POST + serialización JSON.
feature/oled-display	temporal	Driver OLED SSD1306 (I2C).
feature/sdcard-logger	temporal	Datalogger CSV con FatFs sobre SPI.
feature/temp-pwm-control	temporal	Lectura ADC4 + lazo proporcional + PWM.
feature/state-machine	temporal	FSM INIT/ONLINE/OFFLINE/ERROR. global

4. Estructura de Carpetas del Repositorio

```

IoT_Edge_Gateway/
|-- README.md                <- Doc principal (actualizado este avance)
|-- LICENSE
|-- .gitignore
|-- CMakeLists.txt          <- Build root
|-- pico_sdk_import.cmake
|
|-- docs/                    <- Documentacion
|   |-- answers/
|   |   |-- Brando_Chavez_q1.pdf
|   |   |-- Brando_Chavez_q2.pdf
|   |   |-- Brando_Chavez_q3.pdf
|   |   |-- Brando_Chavez_q4.pdf
|   |   |-- Brando_Chavez_q5.pdf
|   |   |-- Brando_Chavez_q6.pdf
|   |-- diagrams/            <- Imagenes y SVGs
|
|-- firmware/                <-Codigo C/C++ del Pico W
|   |-- CMakeLists.txt
|   |-- include/              <- Headers compartidos
|   |-- src/
|   |   |-- main.c            <- super-loop polling + interrupciones
|   |   |-- modbus/
|   |   |-- wifi/
|   |   |-- http/
|   |   |-- oled/
|   |   |-- sdcard/
|   |   |-- temperature/
|   |   |-- pwm_ctrl/
|   |   |-- statemachine/
|
|-- backend/                  <- Django + SQL Server
|   |-- manage.py
|   |-- iot_api/
|   |   |-- views.py
|   |   |-- models.py
|   |   |-- urls.py
|
|-- hardware/                 <- Esquematicos y BOM
|   |-- schematics/
|   |-- BOM.csv

```

5. Flujo de Trabajo (Workflow)

5.1. Comandos típicos durante el desarrollo

```
# 1. Comenzar una nueva feature desde develop
git checkout develop
git pull origin develop
git checkout -b feature/modbus-driver

# 2. Hacer commits atomicos descriptivos
git add src/modbus/modbus_rtu.c
git commit -m "feat(modbus): implementar maestro RTU con DMA"

# 3. Subir la feature
git push -u origin feature/modbus-driver

# 4. Abrir Pull Request en GitHub: feature/* -> develop

# 5. Tras revision, hacer merge y borrar la feature
git checkout develop
git pull
git branch -d feature/modbus-driver

# 6. Liberar a main con tag semantico
git checkout main
git merge --no-ff develop
git tag -a v0.2.0 -m "Release v0.2.0: Modbus + Wi-Fi integrados"
git push origin main --tags
```

6. Actualización de la Documentación

Como parte de este avance, el README.md fue **completamente reescrito** con:

- Badges de tecnología (Pico W, C/C++, Pico SDK, MIT).
- Descripción completa del proyecto con tabla de funciones.
- Diagrama de bloques en formato Mermaid (renderizado por GitHub).
- Listado de RFs y RNFs.
- Escenario de pruebas con ASCII art.
- BOM con costos en COP.
- Estructura del repositorio comentada.
- Instrucciones de compilación (cmake + make).
- Pinout completo (GPIO Pico W → periféricos).
- Tabla de tecnologías y protocolos.

Adicionalmente, todos los PDFs del cuestionario (Q1-Q6) fueron añadidos en docs/answers/, de modo que el profesor pueda acceder a ellos directamente desde el repositorio.

7. Evidencia de Uso del Repositorio

- **Commits frecuentes y atómicos** (un cambio lógico por commit).
- **Mensajes descriptivos** siguiendo Conventional Commits.
- **Pull Requests** para mergear cada feature/* a develop.
- **Tags semánticos** (v0.1.0, v0.2.0...).
- **Issues** para rastrear pendientes (Modbus retry, OLED font, etc.).
- **Documentación viva** en el README.md actualizada en cada merge a main.